

When Only Parts Matter: Efficient Privacy-Preserving Analytics with Fully Homomorphic Encryption

Alexandros Bakas, Dimitrios Schoiniakakis

Nokia Bell Labs, Espoo, Finland

{alexandros.bakas, dimitrios.schoiniakakis}@nokia-bell-labs.com

Keywords: Fully Homomorphic Encryption, Cryptographic Protocols, Region of Interest, Privacy-Preserving Analytics

Abstract: The increasing reliance on cloud-based computation for data-intensive applications raises critical concerns about data confidentiality. Fully Homomorphic Encryption (FHE) provides strong theoretical guarantees by allowing computations over encrypted data, but its high computational cost limits its practicality in large-scale scenarios such as image analysis or matrix-based workloads. In this work, we introduce Π_{ROI} , a hybrid privacy-preserving computation protocol that leverages region-based selective encryption. The core idea is to encrypt only the sensitive Regions of Interest (ROIs) under an FHE scheme, while keeping the remaining, non-sensitive parts of the data in plaintext. This approach achieves end-to-end confidentiality for sensitive regions while significantly improving computational efficiency. We formally define the security of Π_{ROI} through an ideal functionality $\mathcal{F}_{proc}$ and prove that it securely realizes $\mathcal{F}_{proc}$ against a semi-honest cloud service provider under standard cryptographic assumptions (IND-CPA, IND-CCA2, EUF-CMA, and collision-resistance). Experimental evaluation demonstrates that Π_{ROI} offers substantial performance gains in mixed-sensitivity workloads.

1 Introduction

The rapid growth of data-intensive applications in areas such as medical imaging, telecommunications, and industrial monitoring has led to an increasing demand for secure and privacy-preserving computation over sensitive data. In such settings, data owners often outsource computation to cloud service providers (CSPs) to leverage scalable computational resources, while analysts (A) perform advanced data analytics, such as classification or region-based analysis. However, outsourcing data and computations to potentially untrusted cloud infrastructures raises significant privacy concerns, as sensitive information can be exposed.

A natural cryptographic approach to this problem is to compute directly over encrypted data, ensuring that no sensitive information is revealed to the CSP during processing. Fully Homomorphic Encryption (FHE) offers such a capability by extending public-key encryption with an evaluation algorithm that supports arbitrary computations on ciphertexts. Given ciphertexts $c_1, \dots, c_n$ encrypting messages $m_1, \dots, m_n$, the evaluator uses the public key pk_{FHE} to compute

$$c_{out} \leftarrow \text{FHE.Eval}(pk_{FHE}, f, c_1, \dots, c_n),$$

so that decryption reveals $\text{FHE.Dec}(sk_{FHE}, c_{out}) := f(m_1, \dots, m_n)$. This property enables arbitrary computations on encrypted data without revealing any intermediate values, providing a theoretically sound foundation for privacy-preserving outsourcing. Nevertheless, FHE remains computationally expensive

due to noise growth, bootstrapping, and large ciphertext sizes. Such limitations are particularly pronounced in image-based and matrix-based workloads.

To mitigate these costs, several hybrid approaches have been proposed that combine homomorphic and plaintext computation, such as machine learning models that keep parameters in plaintext while operating on FHE-encrypted inputs (Khan et al., 2021; Khan et al., 2023; Gilad-Bachrach et al., 2016). However, most existing methods assume uniformly sensitive data or rely on fixed data partitions, overlooking more realistic scenarios where only certain regions contain sensitive information and hence, incurring unnecessary computational overhead.

While ongoing research in hardware accelerators and dedicated FHE co-processors promises to significantly reduce these costs in the future, such advancements are not yet widely available. In the meantime, it is both possible and necessary to explore architectural and algorithmic optimizations that make FHE more practical on existing hardware. Hybrid cryptographic designs, selective encryption, and mixed-mode computation paradigms can offer immediate and impactful efficiency gains without compromising privacy.

In this work, we address this gap by introducing Π_{ROI} , a hybrid privacy-preserving computation protocol that leverages region-based selective encryption. The core idea is to encrypt only the regions of interest (ROIs), which are identified dynamically by an external secure component under a FHE scheme, while the remaining, non-sensitive parts of the data remain in plaintext. The proposed architecture maintains end-

to-end confidentiality guarantees for sensitive regions while achieving substantial performance gains by performing the majority of operations on unencrypted data.

Our security analysis is carried out in the simulation-based paradigm, where we define an ideal functionality $\mathcal{F}_{\text{proc}}$ that captures the desired privacy goal: the CSP should only learn the size of the input data and the locations of encrypted regions, but nothing about their actual contents or the results of intermediate computations. We formally prove that Π_{ROI} securely realizes $\mathcal{F}_{\text{proc}}$ against a semi-honest adversary modeling the CSP. Specifically, we construct a simulator that can reproduce the CSP’s entire view given only the information allowed by the ideal functionality, showing that the real and ideal executions are computationally indistinguishable. The proof builds on standard cryptographic assumptions: IND-CPA security and circuit privacy of the FHE scheme, IND-CCA2 security of the public-key encryption, EUF-CMA security of the signature scheme, and collision resistance of the hash function. Collectively, these guarantees ensure that Π_{ROI} achieves confidentiality, integrity, and correctness even in the presence of an honest-but-curious cloud provider.

To evaluate the efficiency of our protocol, we implemented a prototype and conducted extensive experiments on image data with varying ROI sizes. Our results demonstrate that the proposed hybrid approach achieves significant computational speedups compared to full homomorphic encryption. For instance, at a ROI ratio of 0.01, our solution achieves up to 84× faster execution on 512×512 images, and maintains a consistent advantage across different image sizes and ROI proportions. These findings confirm that selective encryption guided by dynamic metadata offers a practical pathway toward scalable privacy-preserving analytics.

Contributions: Our main contributions are as follows:

- **Hybrid Privacy-Preserving Framework.** We design Π_{ROI} , a novel protocol that combines region-based selective encryption with homomorphic computation, enabling efficient outsourced processing where only ROIs are encrypted.
- **Formal Security Model and Proof.** We define an ideal functionality $\mathcal{F}_{\text{proc}}$ capturing region-aware privacy guarantees and formally prove that Π_{ROI} securely realizes it under standard assumptions, including IND-CPA and circuit privacy of the FHE scheme, IND-CCA2 of the public-key encryption, EUF-CMA of the signature scheme, and collision resistance of the hash function.
- **Prototype Implementation and Evaluation.** We implement Π_{ROI} and empirically demonstrate that selective encryption achieves substantial performance gains—up to **84× faster execution** compared to full homomorphic processing.

Organization: The remainder of this paper is organized as follows. Section 2 reviews relevant related work. Section 3 introduces the notation and definitions of the cryptographic primitives used throughout the paper. In Section 4, we present the system and threat models. Section 5 constitutes the core contribution of this work, providing a detailed description of our proposed solution, which is followed by a formal security analysis in Section 6 and an experimental evaluation in Section 7. Conclusions are offered in Section 8.

2 Related Work

Homomorphic Encryption and Hybrid Variants

Homomorphic encryption (HE), often dubbed as the “*holy grail of cryptography*”, was first introduced in (Rivest et al., 1978a). Early HE schemes were limited to a single type of operation on ciphertexts. Specifically, the Paillier cryptosystem (Paillier, 1999) supported only additions, while RSA (Rivest et al., 1978b) allowed only multiplications. It was not until 2009 that Gentry introduced the first fully homomorphic encryption (FHE) construction (Gentry, 2009), representing a major milestone in cryptography. With FHE, it became theoretically possible to execute arbitrary computations directly on encrypted data. Despite the conceptual breakthrough, Gentry’s scheme suffered from severe inefficiencies. Nevertheless, it sparked a wave of research, resulting in several follow-up works (Brakerski, 2012; Brakerski et al., 2014; Cheon et al., 2017; Chillotti et al., 2020; Fan and Vercauteren, 2012) that aimed to overcome these limitations and design more practical and efficient FHE schemes. However, no matter the advancements in the field, FHE still remains highly impractical for large scale deployment. In an attempt to improve this, several lines of work have turned to the fascinating topic of Hybrid Homomorphic Encryption (HHE) that was first introduced in (Naehrig et al., 2011), but the first formal definitions appeared in (Dobraunig et al., 2023) and the first complete protocol in (Bakas et al., 2022). In an HHE scheme, the user encrypts data locally with a symmetric cipher and the symmetric key with FHE, then outsources both to a CSP. The CSP runs the FHE decryption circuit to convert the symmetric ciphertexts into FHE ciphertexts, offloading heavy computations from the user. However, existing works treat all data as equally sensitive, ignoring that some parts may require stronger protection.

Cryptographic ROI protection To the best of our knowledge, only two works leverage strong cryptographic guarantees to protect ROIs within a file. In (Akçay et al., 2025), the authors propose a solution where an ML model first detects ROIs in images, which are then encrypted using Attribute-Based Encryption (ABE)—a form of public-key encryption that

enforces fine-grained access control based on user attributes. This ensures that only authorized parties can decrypt ROIs according to their access rights. Similarly, the authors in (Nuoskala et al., 2026) extend this approach to videos. However, despite their promise, these works do not enable any computation on the encrypted ROIs, as such functionality is not supported by ABE.

Non-cryptographic ROI protection A common practice for protecting sensitive information in images is to use blurring or pixel-based redaction (Orecondy et al., 2018). These methods are easy to apply and computationally lightweight, making them suitable for large-scale or real-time processing pipelines. However, they suffer from several key limitations. First, computations or further analysis on the ROIs become impossible once the sensitive regions are obscured. Second, the protection they offer is purely visual—an attacker may still recover the redacted content using modern reconstruction or inpainting techniques (Bland et al., 2022). Finally, such approaches provide no formal security guarantees, as their effectiveness depends heavily on the level of distortion applied and the attacker’s prior knowledge or available computational tools.

3 Preliminaries

3.1 Notation

Notation: The output y of a probabilistic algorithm A_P on input x is denoted by $y \leftarrow A_P(x)$, while the output y of a deterministic algorithm A_D on input x is denoted by $y := A_D(x)$. A PPT adversary $\mathcal{A}$ is a randomized algorithm for which there exists a polynomial $p(z)$ such that for all input z , the running time of $\mathcal{A}(z)$ is bounded by $p(|z|)$. A function $\text{negl}(\cdot)$ is called negligible if $\forall c \in \mathbb{N}, \exists \epsilon_0$ such that $\forall \epsilon > \epsilon_0 : \text{negl}(\epsilon) < \epsilon^{-c}$.

3.2 Cryptographic Primitives

Definition 1 (Public-Key Encryption). A public-key encryption scheme PKE with message space $\mathcal{M}$ and ciphertext space $\mathcal{C}$ consists of three polynomial-time algorithms

$$\text{PKE} = (\text{PKE.Gen}, \text{PKE.Enc}, \text{PKE.Dec}),$$

defined as follows:

- **Key Generation:** The key generation algorithm PKE.Gen takes a security parameter λ and outputs a public/private key pair (pk, sk) :

$$(pk, sk) \leftarrow \text{PKE.Gen}(1^\lambda).$$

- **Encryption:** The encryption algorithm PKE.Enc takes as input the public key pk and a message $m \in \mathcal{M}$, and outputs a ciphertext $c \in \mathcal{C}$:

$$c \leftarrow \text{PKE.Enc}(pk, m).$$

- **Decryption:** The decryption algorithm PKE.Dec takes as input the private key sk and a ciphertext $c \in \mathcal{C}$, and outputs a message $m \in \mathcal{M}$:

$$m := \text{PKE.Dec}(sk, c).$$

Finally, the correctness property requires that $\text{PKE.Dec}(sk, \text{PKE.Enc}(pk, m)) = m$ for all (pk, sk) and $m \in \mathcal{M}$.

Definition 2 (Fully Homomorphic Encryption). A fully homomorphic encryption (FHE) scheme is a public-key encryption scheme FHE with message space $\mathcal{M}$ and ciphertext space $\mathcal{C}$ that extends PKE with:

- **Key Generation:** The key generation algorithm outputs a public/private key pair along with an evaluation key:

$$(pk_{\text{FHE}}, sk_{\text{FHE}}, evk_{\text{FHE}}) \leftarrow \text{FHE.KeyGen}(1^\lambda).$$

Both pk_{FHE} and evk_{FHE} are public; for simplicity, we treat evk_{FHE} as part of pk_{FHE} in the rest of the paper, and refer only to pk_{FHE} when discussing public keys.

- **Encryption and Decryption:** Same as in PKE, with keys $(pk_{\text{FHE}}, sk_{\text{FHE}})$.
- **Evaluation:** The evaluation algorithm FHE.Eval is a PPT algorithm that takes as input the public key pk_{FHE} , a function (or circuit) f , and a sequence of inputs consisting of ciphertexts and optionally plaintexts. Given ciphertexts $c_1, \dots, c_n \in \mathcal{C}$ and plaintexts $m_1, \dots, m_\ell \in \mathcal{M}$, it outputs an evaluated ciphertext

$$c_{\text{out}} \leftarrow \text{FHE.Eval}(pk_{\text{FHE}}, f, c_1, \dots, c_n, m_1, \dots, m_\ell),$$

satisfying

$$\text{FHE.Dec}(sk_{\text{FHE}}, c_{\text{out}}) = f(m'_1, \dots, m'_n, m_1, \dots, m_\ell),$$

where m'_i denotes the plaintext encrypted in c_i . Correctness ensures that decrypting c_{out} yields the same result as applying f directly to the corresponding plaintext inputs.

4 Architecture & Threat Model

4.1 Architecture

- **Function Owner (FO):** FO is responsible for providing the computation parameters W to the CSP. FO ensures that these parameters are correctly formatted, securely transmitted, and authenticated so that CSP can use them in the computations. FO does not handle raw input data or perform any computation itself.
- **Cloud Service Provider (CSP):** CSP performs the outsourced computations on the data it receives, which may include a combination of plaintext and encrypted components. CSP also stores the computation parameters W received from FO

and ensures that all operations are carried out correctly. In the semi-honest model, CSP follows the protocol honestly but may try to learn additional information from the messages it receives.

- **Metadata Authority (MA):** MA processes the raw plaintext input to extract relevant metadata, such as the coordinates, size, and shape of regions of interest (ROI). This metadata guides the encryption of sensitive regions by the Analyst and the computation splitting performed by the CSP. MA's output may include limited information about the ROIs, which is allowed to be revealed to the CSP. For the rest of the paper we assume that MA resides on the Analyst's local machine.
- **Analyst (A):** Analyst A is responsible for encrypting the sensitive parts of the input (ROI) according to the metadata provided by MA. Analyst A prepares the computation request, sends it to CSP, and upon receiving the result, decrypts it to obtain the final output. Analyst A ensures that sensitive data remains confidential while still allowing the computation to be performed on encrypted regions.

MA is assumed to be a fully trusted component. In practice, MA can be either a local service integrated into the Analyst's workflow or a third party component running in a secure enclave. Its role is to process raw data to extract non-sensitive metadata (e.g., coordinates and dimensions of regions of interest) that guide selective encryption and computation partitioning. When the Analyst is a domain expert, such as a medical professional capable of manually selecting regions of interest, the MA may simply act as a supporting tool. However, for non-technical Analysts or automated clients, MA enables seamless pre-processing without exposing raw data outside a trusted environment. This design ensures both usability and strong privacy guarantees.

4.2 Threat Model

Our threat model assumes a *semi-honest* (honest-but-curious) CSP, which faithfully follows the protocol specification but may try to learn additional information from the messages it receives. The CSP may observe all stored data, metadata, and encrypted inputs, and attempt to infer sensitive information.

In addition to the semi-honest CSP, the protocol is designed to withstand *active attacks*, where an adversary $\mathcal{A}$ may attempt to forge, replay, or tamper with messages. We protect against these attacks using EUF-CMA-secure signatures, collision-resistant hash functions, and timestamps to ensure message integrity and freshness. For example, an active adversary could attempt to impersonate the Function Owner (FO) or the Analyst (A) or replay an old computation request, but such attempts would be detected and rejected by the verification steps in the protocol.

This threat model captures realistic cloud deployment scenarios, where the CSP may be curious about

the computation but is prevented from deviating arbitrarily without being detected, and where cryptographic mechanisms provide resilience against fully active adversaries.

5 Protocol

We are now ready to proceed with the formal description of our protocol Π_{ROI} . Π_{ROI} consists of four polynomial-time sub-protocols such that $\Pi_{ROI} = (\text{Setup}, \text{Outsource}, \text{Compute}, \text{Return})$.

Building Blocks The main building blocks of Π_{ROI} are:

- An IND-CCA2 secure public-key encryption scheme PKE;
- An IND-CPA secure fully homomorphic encryption scheme FHE;
- An EUF-CMA secure signature scheme Sign;
- A collision-resistant hash function H ;
- A synchronized clock shared between all entities.

Setup. Each entity from the described architecture obtains a public/private key pair (pk, sk) for a CCA2-secure public-key cryptosystem. The public keys are published while the private keys are kept secret. Moreover, the entities generate a signing and verification key pair (sig, ver) for an EUF-CMA secure signature scheme. Finally, the analyst A generates a public/private key pair for a FHE scheme (pk_{FHE}, sk_{FHE}) as well as an evaluation key evk_{FHE} . The evaluation key can be seen as a component of the public key pk_{FHE} . Below we provide the list of generated keys.

- **FO keys:** A public private key pair (pk_{FO}, sk_{FO}) for a public-key encryption scheme PKE and a signing/verification key pair (sig_{FO}, ver_{FO}) for a signature scheme S .
- **CSP keys** A public private key pair (pk_{CSP}, sk_{CSP}) for a public-key encryption scheme PKE and a signing/verification key pair (sig_{CSP}, ver_{CSP}) for a signature scheme S .
- **Analyst A keys:** A public private key pair (pk_A, sk_A) for a public-key encryption scheme PKE and a signing/verification key pair (sig_A, ver_A) for a signature scheme S . Moreover, A generates a public/private key pair pk_{FHE}, sk_{FHE} for a fully homomorphic encryption scheme FHE.

Outsourcing Computations. FO first defines a sequence of operations that will be applied to the encrypted data. That is, FO defines a matrix $W = [w_1, \dots, w_n]$, representing the parameters of one or more linear layers. As a next step, FO computes $c_W \leftarrow \text{Enc}(pk_{CSP}, W)$ and outsources it to the CSP via:

$$m_1 = \langle t_1, c_W, \sigma_{sig_{FO}}(H_1) \rangle$$

where t_1 is a timestamp and $H_1 = H(t_1 \parallel c_W)$. Upon reception, the CSP first verifies the freshness of the message by checking the timestamp, and subsequently its integrity and authenticity by computing $- := V(\text{ver}_{FO}, \sigma_{\text{sig}_{FO}})$. If the verification passes, the CSP decrypts c_W using its private key sk_{CSP} to receive W , and then stores it.

Computations. Analyst A runs an image through MA to obtain the coordinates of the ROIs. For each image, the ROIs may differ in location, size, or shape. We assume that the output of MA is some metadata mtd. As a next step, A encrypts only the pixels within the ROI:

$$c_{\text{ROI}} \leftarrow \text{FHE.Enc}(\text{pk}_{\text{FHE}}, \text{ROI})$$

while the rest of the image remains in plaintext. The encrypted ROI and plaintext non-ROI are concatenated to form a partially encrypted image vector $c_{\text{img}} = (c_{\text{ROI}}, \text{non-ROI})$. Finally, A further encrypts mtd by computing $c_{\text{mtd}} \leftarrow \text{PKE.Enc}(\text{pk}_{\text{CSP}}, \text{mtd})$ and sends the partially encrypted image and the metadata to the CSP via:

$$m_2 = \langle t_2, c_{\text{img}}, c_{\text{mtd}}, \sigma_{\text{sig}_A}(H_2) \rangle$$

where t_2 is a timestamp and $H_2 = H(t_2 \parallel c_{\text{img}} \parallel c_{\text{mtd}})$.

Upon reception, the CSP verifies the freshness and integrity of the message as before. If the verification succeeds, the CSP proceeds with decrypting mtd by computing $\text{mtd} := \text{PKE.DEC}(\text{sk}_{\text{CSP}}, c_{\text{mtd}})$.

Using the ROI coordinates (retrieved by mtd), the CSP dynamically splits W into

$$W = [W_{\text{ROI}} \quad W_{\text{non-ROI}}]$$

and performs the computations separately on the encrypted and plaintext parts. Concretely, the CSP performs dot products between the encrypted ROI and W_{ROI} , and between the plaintext non-ROI and $W_{\text{non-ROI}}$ ¹. The final result is obtained by combining the two contributions:

$$c_{\text{out}} = W_{\text{ROI}} \cdot c_{\text{ROI}} + W_{\text{non-ROI}} \cdot X_{\text{non-ROI}}.$$

Return After performing the homomorphic and plaintext computations described above, the CSP returns the result to A via:

$$m_3 = \langle t_3, c_{\text{out}}, \sigma_{\text{sig}_{\text{CSP}}}(H_3) \rangle,$$

where c_{out} is the partially or fully encrypted output of the computation, t_3 is a timestamp, and $H_3 = H(t_3 \parallel c_{\text{out}})$ ensures message integrity. After verifying the integrity of m_3 , A proceeds by decrypting c_{out} and recovering the output out in plaintext.

¹Optionally followed by further linear operations (e.g., fully connected CNN layers)

6 Security proof against a semi-honest CSP

Before presenting the formal proof, we outline its structure and intuition. The proof proceeds in the simulation-based paradigm for a *semi-honest cloud service provider (CSP)*. We define an ideal functionality $\mathcal{F}_{\text{proc}}$ that captures the intended behavior of the protocol. That is, performing encrypted operations on outsourced data according to authorized instructions, while revealing only the permitted leakage (e.g., data size, access pattern, or evaluation result). The goal is to show that any information learned by a *semi-honest CSP* in the real execution can be simulated using only this leakage.

The *simulator* $\mathcal{S}$ uses the public keys, the structure of ciphertexts, and the leakage function to generate a view indistinguishable from the CSP's real view. The argument relies on the semantic security of the underlying encryption scheme, the unforgeability of signatures, and the circuit privacy property to ensure that the evaluated ciphertexts (e.g., c_{out}) do not leak additional information about the plaintexts or the function. Freshness and integrity of requests are enforced via signed timestamps, whose forgery would contradict EUF-CMA security.

In the simulation-based paradigm, the *ideal functionality* $\mathcal{F}_{\text{proc}}$ serves as the abstract specification of the protocol's intended behavior, defining which data is kept secret and what information may be leaked (e.g., ROI coordinates, size, or evaluation results). The *simulator* $\mathcal{S}$ is a hypothetical algorithm used in the proof to generate a view for the semi-honest CSP that is indistinguishable from its real view in the actual protocol, using only the information allowed by $\mathcal{F}_{\text{proc}}$. The existence of such a simulator demonstrates that the adversary learns nothing beyond the permitted leakage, showing that the real protocol securely realizes $\mathcal{F}_{\text{proc}}$.

This establishes that the protocol securely realizes $\mathcal{F}_{\text{proc}}$ against a *semi-honest CSP*.

Ideal functionality $\mathcal{F}_{\text{proc}}$

$\mathcal{F}_{\text{proc}}$ interacts with FO, Analyst A , and an adversary (modeling a semi-honest CSP) as follows:

- **Input from FO:** matrix W . $\mathcal{F}_{\text{proc}}$ stores W internally.
- **Input from Analyst A :** $(X_{\text{non-ROI}}, \text{ROI})$ and metadata mtd. ROI is kept secret; $X_{\text{non-ROI}}$ is public. $\mathcal{F}_{\text{proc}}$ may leak allowed information (ROI coordinates, size) to the adversary.
- **Computation:** $\text{out} := W_{\text{ROI}} \cdot \text{ROI} + W_{\text{non-ROI}} \cdot X_{\text{non-ROI}}$.
- **Output:** return out to A ; optionally leak allowed size/shape information to the adversary.

Security statement: for every PPT adversary $\mathcal{A}$ that corrupts CSP in the real protocol, there exists a PPT simulator $\mathcal{S}$ such that the real-world view of $\mathcal{A}$ is computationally indistinguishable from the simulated view produced by $\mathcal{S}$ interacting with $\mathcal{F}_{\text{proc}}$ and the honest parties.

Leakage Function We now proceed by providing a formal definition of the leakage function $\mathcal{L}$ associated with Π_{ROI} .

Definition 3 (Leakage Function $\mathcal{L}$). *Let $\mathcal{L}$ denote the leakage function associated with the protocol. For each input tuple $(W, X_{\text{non-ROI}}, \text{ROI}, \text{mtd})$, the leakage function outputs all information that is intentionally revealed to the semi-honest CSP during the execution of the protocol. Formally,*

$$\mathcal{L}(W, X_{\text{non-ROI}}, \text{ROI}, \text{mtd}) = (W, X_{\text{non-ROI}}, f(\text{mtd})),$$

where:

- W is the matrix of linear layer parameters (stored by CSP in the real protocol),
- $X_{\text{non-ROI}}$ is the plaintext portion of the input image,
- $f(\text{mtd})$ is a function that extracts the allowed leakage from the metadata (e.g., ROI coordinates, ROI size or shape),

Thus, $\mathcal{L}$ captures exactly the information that a semi-honest CSP may observe, and the security proof will show that no additional information about ROI or other sensitive inputs is leaked.

6.1 Formal Proof

Theorem 1. *Under the assumptions above (IND-CPA for PKE, semantic security + circuit privacy for FHE, and EUF-CMA for SIG), Π_{ROI} securely realizes $\mathcal{F}_{\text{proc}}$ against any semi-honest adversary that corrupts the CSP*

Proof. We present a PPT simulator $\mathcal{S}$ for a semi-honest CSP and a sequence of hybrids arguing indistinguishability from the real execution. The simulator $\mathcal{S}$ receives exactly the leakage that $\mathcal{F}_{\text{proc}}$ specifies: in particular, it obtains W , the plaintext non-ROI data, and the metadata (ROI coordinates/shape) if those are modeled as leaked. Using only that leakage, $\mathcal{S}$ must produce a view indistinguishable from a real CSP view.

CSP's real view

In the real protocol the CSP's view consists of:

1. The message $m_1 = \langle t_1, c_W, \sigma_{\text{sig}_{FO}}(H_1) \rangle$ from FO (CSP decrypts c_W to obtain W).
2. The stored W .
3. The message $m_2 = \langle t_2, c_{\text{img}}, c_{\text{mtd}}, \sigma_{\text{sig}_A}(H_2) \rangle$ from Analyst A , where $c_{\text{img}} = (c_{\text{ROI}}, \text{non-ROI})$.
4. The homomorphic evaluation process on c_{ROI} with W_{ROI} and the plaintext computation on non-ROI with $W_{\text{non-ROI}}$, yielding c_{out} .
5. The output message $m_3 = \langle t_3, c_{\text{out}}, \sigma_{\text{sig}_{CSP}}(H_3) \rangle$ (sent to A).
6. Any internal randomness and transcript of the above.

Simulator $\mathcal{S}$ (high-level). $\mathcal{S}$ is given leakage $\mathcal{L} = (W, \text{non-ROI}, \text{mtd})$ by $\mathcal{F}_{\text{proc}}$. Using $\mathcal{L}$ it must simulate all messages and the homomorphic computation record. The simulator works as follows:

1. Simulate receipt of m_1 : $\mathcal{S}$ constructs a fresh timestamp t_1 , picks a random string c_W^* of the same length as a real encryption of W , computes $H_1^* = H(t_1 \parallel c_W^*)$, and uses a simulated signature value $\sigma_{\text{sig}_{FO}}^*$ (see discussion below) to form $m_1^* = \langle t_1, c_W^*, \sigma_{\text{sig}_{FO}}^*(H_1^*) \rangle$. Because the real CSP would decrypt c_W to W , $\mathcal{S}$ additionally places W in its internal simulated storage for subsequent computation.
2. Simulate receipt of m_2 : $\mathcal{S}$ obtains non-ROI and mtd from the leakage $\mathcal{L}$. It selects a timestamp t_2 and produces a simulated FHE ciphertext c_{ROI}^* (a random string of proper length) and a simulated PKE ciphertext c_{mtd}^* (a random IND-CPA encryption-length string). It sets $c_{\text{img}}^* = (c_{\text{ROI}}^*, \text{non-ROI})$. It forms $H_2^* = H(t_2 \parallel c_{\text{img}}^* \parallel c_{\text{mtd}}^*)$ and a simulated signature $\sigma_{\text{sig}_A}^*$ and outputs $m_2^* = \langle t_2, c_{\text{img}}^*, c_{\text{mtd}}^*, \sigma_{\text{sig}_A}^*(H_2^*) \rangle$.
3. Simulate the evaluation: $\mathcal{S}$ obtains W , non-ROI, and mtd from the leakage and (knowing the ROI coordinates from mtd) computes the true arithmetic result $\text{out} = W_{\text{ROI}} \cdot \widetilde{\text{ROI}} + W_{\text{non-ROI}} \cdot \text{non-ROI}$ where $\widetilde{\text{ROI}}$ is an arbitrary placeholder consistent with simulated ciphertext length (the simulator does not know the real ROI plaintext). Using circuit privacy of FHE, $\mathcal{S}$ generates a simulated evaluated ciphertext c_{out}^* that is indistinguishable from a real FHE evaluation on an encryption of ROI under pk_{FHE} producing an encryption of out . (Concretely, $\mathcal{S}$ may sample a ciphertext consistent with the distribution of evaluated ciphertexts, or use the FHE simulator guaranteed by circuit-privacy.)
4. Form $m_3^* = \langle t_3, c_{\text{out}}^*, \sigma_{\text{sig}_{CSP}}^*(H_3^*) \rangle$ analogously and output it.

Remarks about simulated signatures: since $\mathcal{S}$ does not hold honest parties' signing keys, the signatures σ^* are simulated values consistent with the signature length and format. In the semi-honest model, $\mathcal{A}$ (the adversary controlling CSP) never requests honest parties to sign arbitrary messages, hence EUF-CMA security ensures the adversary cannot produce a valid forgery, and simulated signatures (which look like random strings) are indistinguishable from real signatures in the adversary's view under the standard assumption that signature tags do not leak additional structure beyond their length. If one needs a fully formal treatment, this step is captured by a standard EUF-CMA reduction: any distinguisher between simulated and real signatures leads to forging advantage against EUF-CMA.

Hybrid sequence. We now show indistinguishability between the real view and the simulated view via a short hybrid argument.

Hybrid $\mathcal{H}_0$: the real execution.

Hybrid $\mathcal{H}_1$: Identical to $\mathcal{H}_0$ but replace $c_{\text{mtd}} = \text{PKE.Enc}(\text{pk}_{\text{CSP}}, \text{mtd})$ in message m_2 with an encryption of 0^ℓ (or a random string of the same length). By IND-CPA security of PKE, no PPT distinguisher (including $\mathcal{A}$) can distinguish $\mathcal{H}_1$ from $\mathcal{H}_0$ except with advantage at most ϵ_{PKE} .

Hybrid $\mathcal{H}_2$: Replace the FHE ciphertext $c_{\text{ROI}} = \text{FHE.Enc}(\text{pk}_{\text{FHE}}, \text{ROI})$ with an encryption of 0 (or a ciphertext sampled from the ciphertext space), leaving the plaintext non-ROI unchanged. By IND-CPA security (semantic security) of FHE, no PPT distinguisher can distinguish $\mathcal{H}_2$ from $\mathcal{H}_1$ except with advantage at most $\epsilon_{\text{FHE-enc}}$.

Hybrid $\mathcal{H}_3$: Replace the evaluated ciphertext c_{out} produced by homomorphic evaluation with a simulated evaluated ciphertext c_{out}^* that depends only on the allowed leakage (i.e., its distribution is independent of the hidden ROI plaintext beyond the leakage). Circuit-privacy of FHE guarantees that the distribution of evaluated ciphertexts can be simulated given the public circuit and the allowed leakage; therefore, no PPT distinguisher can distinguish $\mathcal{H}_3$ from $\mathcal{H}_2$ except with advantage at most $\epsilon_{\text{FHE-eval}}$.

Hybrid $\mathcal{H}_4$: Replace the PKE ciphertext c_W in m_1 with a random string of the same length c_W^* . Since in the real protocol the CSP immediately decrypts c_W to obtain W (and we model that in the leakage), replacing the ciphertext with a random-string placeholder is indistinguishable under IND-CPA of the scheme used for encrypting W . Thus no distinguisher can separate $\mathcal{H}_4$ from $\mathcal{H}_3$ except with advantage ϵ_{PKE} (re-using same bound).

Finally, observe that $\mathcal{H}_4$ is exactly the distribution produced by the simulator $\mathcal{S}$ (which uses random-looking ciphertexts/signatures and simulated evaluated ciphertexts consistent with leakage). Therefore the real view and the simulated view are computation-

ally indistinguishable with total distinguishing advantage bounded by the sum of primitive advantages:

$$\epsilon_{\text{total}} \leq 2\epsilon_{\text{PKE}} + \epsilon_{\text{FHE-enc}} + \epsilon_{\text{FHE-eval}} + \epsilon_{\text{SIG}},$$

where ϵ_{SIG} captures any (negligible) difference arising from simulated vs real signatures (reducible to EUF-CMA breaking advantage). However, it is a standard result in real analysis, that the finite sum of negligible functions, is still negligible. Hence, ϵ_{total} is negligible. $\square$

For any PPT adversary $\mathcal{A}$ corrupting the CSP, we constructed a PPT simulator $\mathcal{S}$ that, given the allowed leakage, produces a simulated view computationally indistinguishable from the real view. Hence the protocol securely realizes $\mathcal{F}_{\text{proc}}$ against semi-honest CSPs, assuming IND-CPA security of PKE, semantic security + circuit-privacy of FHE, and EUF-CMA security of signatures.

7 Experimental Evaluation

We evaluate the performance of our proposed hybrid homomorphic computation framework by comparing it against a conventional fully homomorphic approach across two complementary experiments: (i) computation-time benchmarking for encrypted matrix operations, and (ii) encryption and decryption cost analysis. All experiments were implemented in Python using the TenSEAL library (CKKS scheme) and executed on a workstation with an Intel Core i5-1245U CPU (10 cores, 12 threads, 1.6 GHz) and 32 GB of RAM running Windows 11.

Setup. We consider grayscale image vectors of size $n = \text{IMG_SIZE}^2$, where $\text{IMG_SIZE} \in \{64, 128, 256, 512\}$. Each image is divided into a sensitive *region of interest* (ROI), defined by a ratio $\rho \in \{0.01, 0.05, 0.10, 0.25\}$ representing the fraction of pixels treated as sensitive and thus encrypted, while the remaining non-ROI pixels are processed in plaintext. To select meaningful ROI ratios, we refer to the BRATS brain tumor segmentation dataset, where the automatically segmented tumor regions occupy on average about 1.6% of the total MRI image area. This observation motivates the use of small ρ values (e.g., 0.01) as realistic cases for localized sensitivity.

For our experiments, we generate synthetic grayscale data rather than using real MRIs. This choice avoids potential privacy concerns, facilitates reproducibility, and mitigates computational challenges associated with the large 3D volumes of MRI data, which are prohibitively expensive to process under the CKKS scheme using current implementations such as TenSEAL. Since our goal is to measure computational and encryption overheads—quantities that depend primarily on data dimensionality rather than semantic content—synthetic images provide an appropriate and ethically neutral substitute. All synthetic

images were randomly generated with pixel intensities uniformly distributed in $[0, 1]$.

We employ the CKKS scheme with polynomial modulus degree 16384, coefficient modulus sizes $[60, 40, 40, 40, 40, 60]$, and a global scale of 2^{40} . Each configuration was repeated 10 times, and all reported values represent averages across these runs.

Dataset Justification. While real-world MRI datasets exhibit complex spatial and intensity distributions, the computational characteristics of homomorphic encryption primarily depend on data dimensionality and the proportion of encrypted inputs. Therefore, the performance trends observed on synthetic grayscale images are expected to generalize to real MRI data. By abstracting away domain-specific content, our evaluation isolates the cryptographic and computational aspects of the proposed hybrid scheme, ensuring that the measured performance differences reflect encryption-related costs rather than dataset-specific artifacts.

Experiment 1: Server side Performance. The first experiment measures the total runtime of two linear-computation rounds on encrypted image data. Each image is represented as a vector of length $N = \text{IMG.SIZE}^2$, and each round consists of matrix-vector multiplications simulating computations typically found in machine learning workloads.

Specifically, in **Round 1**, a matrix $M_1 \in \mathbb{R}^{20 \times N}$ with randomly generated entries is multiplied by the encrypted image vector in the fully homomorphic (FHE) case. In the hybrid variant, only the ROI pixels of the image are encrypted, while the remaining pixels are processed in plaintext; the server computes the dot product between the encrypted ROI and the corresponding matrix columns, and adds the result of a standard dot product on the plaintext non-ROI portion. The choice of 20 rows in M_1 provides a moderate computational load that is sufficient to highlight the performance differences between FHE and hybrid approaches while keeping the benchmark tractable.

In **Round 2**, the outputs of Round 1 are treated as vectors of intermediate activations. A second matrix $M_2 \in \mathbb{R}^{10 \times 20}$ is multiplied by these vectors using a block-wise approach. All intermediate values are encrypted in the fully homomorphic case, and aggregated using pairwise summation of partial products. In the hybrid variant, ROI-encrypted components are multiplied homomorphically, while the remaining intermediate values are multiplied in plaintext and summed with the encrypted contributions. The 10 rows of M_2 provide a realistic simulation of a smaller fully connected layer in a neural network, allowing us to demonstrate the impact of encryption on successive linear layers without excessive computational cost.

This setup—two consecutive linear layers with carefully chosen matrix dimensions—provides a practical simulation of machine learning computations on

encrypted data, illustrating both the computational cost of fully homomorphic encryption and the efficiency gains achievable with the hybrid approach. Table 1 shows representative runtimes for the smallest and largest ROI ratios for each image size, and Figure 1 depicts the total speedup versus ρ .

Across all image sizes, the hybrid approach consistently achieves significant acceleration over the fully homomorphic baseline. For small ROIs ($\rho = 0.01$), the total speedup reaches up to $\times 84$ for 512×512 images and $\times 60$ for 256×256 , while maintaining correctness and privacy for the sensitive region. Even for larger ROIs ($\rho = 0.25$), the hybrid method remains approximately $4\times$ faster on average. These gains stem from replacing a large fraction of ciphertext-plaintext multiplications with inexpensive plaintext operations, demonstrating that hybrid processing can substantially reduce homomorphic overheads in data with localized sensitivity.

Experiment 2: Encryption and Decryption Costs.

To complement the above, we benchmark the cost of CKKS encryption and decryption under the same $(\text{IMG.SIZE}, \rho)$ configurations. Table 2 presents the average times per image. Results show that encryption dominates decryption time in all cases, and both scale approximately linearly with the number of encrypted slots. When only a small ROI is encrypted, the reduction in ciphertext size yields major time savings: for instance, at $\rho = 0.01$, encryption is $60\times$ faster and decryption over $50\times$ faster compared to full-image encryption for 512×512 images. This reduction directly translates to lower communication costs when outsourcing encrypted data.

Discussion. Together, the results highlight the computational benefits of selectively encrypting sensitive subregions while retaining privacy guarantees. The hybrid approach effectively bridges the gap between practicality and security, enabling near-real-time performance for large-scale encrypted workloads in scenarios such as medical imaging or industrial monitoring.

8 Conclusion

In this paper, We presented Π_{ROI} , a hybrid privacy-preserving computation protocol designed for mixed-sensitivity data. By selectively applying FHE only to sensitive regions of interest and processing the remaining data in plaintext, Π_{ROI} achieves a significant reduction in computational overhead while preserving strong security guarantees. Our formal proof establishes that the protocol securely realizes an ideal functionality $\mathcal{F}_{\text{proc}}$ against a semi-honest cloud service provider, relying on the security of standard cryptographic primitives. This design demonstrates that a fine-grained separation between encrypted and unencrypted computation can yield both efficiency and

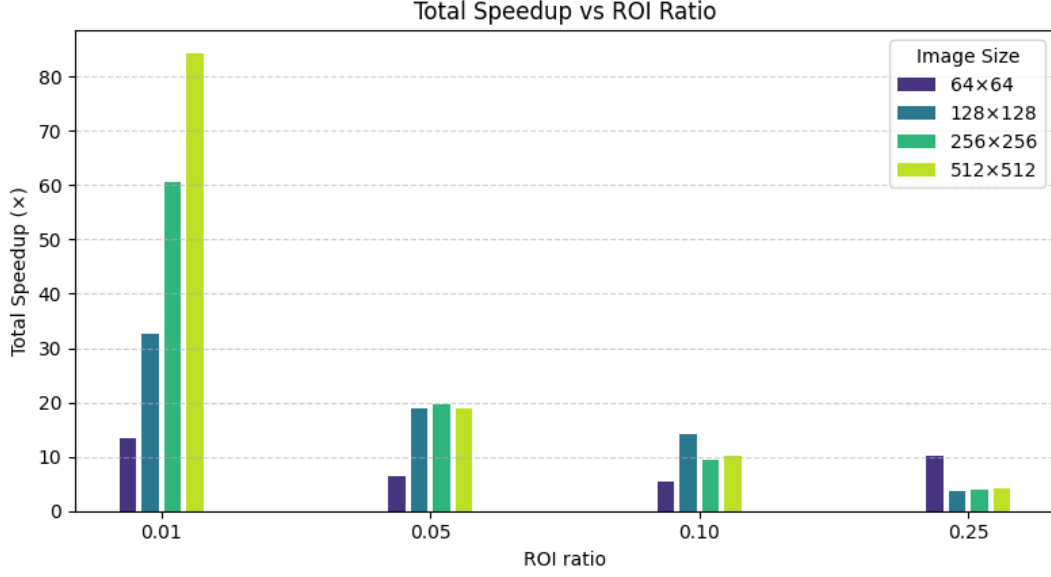


Figure 1: Total speedup (Full / Hybrid) as a function of ROI ratio for different image sizes.

IMG Size	ROI Ratio	Full Comp. (s)	Hybrid Comp. (s)	Speedup	Full Enc. (s)	ROI Enc. (s)	Enc. Speedup
64	0.01	19.86	1.49	13.33	0.032	0.009	3.56
64	0.25	23.99	2.38	10.07	0.045	0.017	2.65
128	0.01	88.63	2.71	32.67	0.146	0.016	9.13
128	0.25	93.26	24.79	3.76	0.110	0.040	2.75
256	0.01	276.56	4.57	60.55	0.395	0.015	26.3
256	0.25	269.62	66.78	4.04	0.399	0.113	3.53
512	0.01	1048.29	12.44	84.26	1.809	0.025	72.36
512	0.25	1143.31	280.88	4.07	1.905	0.473	4.03

Table 1: Comparison of computation and encryption performance across image sizes and ROI ratios.

Image Size	ROI Ratio	Full Enc. (s)	ROI Enc. (s)	Full Dec. (s)	ROI Dec. (s)	Total Speedup
64	0.010	0.0321	0.0093	0.0114	0.0009	4.27x
64	0.050	0.0356	0.0107	0.0147	0.0022	3.90x
64	0.100	0.0394	0.0154	0.0161	0.0045	2.80x
64	0.250	0.0452	0.0170	0.0169	0.0027	3.14x
128	0.010	0.1461	0.0159	0.0547	0.0057	9.30x
128	0.050	0.1175	0.0170	0.0435	0.0045	7.46x
128	0.100	0.1162	0.0143	0.0376	0.0048	8.06x
128	0.250	0.1102	0.0404	0.0423	0.0112	2.95x
256	0.010	0.3955	0.0145	0.1364	0.0020	32.21x
256	0.050	0.4090	0.0272	0.1423	0.0083	15.53x
256	0.100	0.3996	0.0484	0.1361	0.0165	8.26x
256	0.250	0.3986	0.1126	0.1381	0.0395	3.53x
512	0.010	1.8095	0.0252	0.5538	0.0106	65.92x
512	0.050	1.7257	0.0908	0.5822	0.0337	18.53x
512	0.100	1.6328	0.1630	0.5527	0.0585	9.87x
512	0.250	1.9049	0.4728	0.6181	0.1527	4.03x

Table 2: Client-side encryption and decryption times with total speedup

provable security. Future work includes extending the model to malicious adversaries, optimizing the secure component’s ROI detection process, and evaluating the protocol’s scalability in real-world multi-party cloud environments.

REFERENCES

- Akcaay, M. H., Atli, B. G., Rao, S. P., and Bakas, A. (2025). From see to shield: MI-assisted fine-grained access control for visual data.
- Bakas, A., Frimpong, E., and Michalas, A. (2022). Symmetrical disguise: Realizing homomorphic encryption services from symmetric primitives. In *International*

- Conference on Security and Privacy in Communication Systems*, pages 353–370. Springer.
- Bland, M., Iyer, A., and Levchenko, K. (2022). Story beyond the eye: glyph positions break pdf text redaction. *arXiv preprint arXiv:2206.02285*.
- Brakerski, Z. (2012). Fully homomorphic encryption without modulus switching from classical gapsvp. In *Annual cryptology conference*, pages 868–886. Springer.
- Brakerski, Z., Gentry, C., and Vaikuntanathan, V. (2014). (leveled) fully homomorphic encryption without bootstrapping. *ACM Transactions on Computation Theory (TOCT)*, 6(3):1–36.
- Cheon, J. H., Kim, A., Kim, M., and Song, Y. (2017). Homomorphic encryption for arithmetic of approximate numbers. In *International conference on the theory and application of cryptology and information security*, pages 409–437. Springer.
- Chillotti, I., Gama, N., Georgieva, M., and Izabachène, M. (2020). Tfhe: fast fully homomorphic encryption over the torus. *Journal of Cryptology*, 33(1):34–91.
- Dobraunig, C., Grassi, L., Helminger, L., Rechberger, C., Schofnegger, M., and Walch, R. (2023). Pasta: A case for hybrid homomorphic encryption. *IACR Transactions on Cryptographic Hardware and Embedded Systems*, 2023(3):30–73.
- Fan, J. and Vercauteren, F. (2012). Somewhat practical fully homomorphic encryption. *Cryptology ePrint Archive*.
- Gentry, C. (2009). Fully homomorphic encryption using ideal lattices. In *Proceedings of the forty-first annual ACM symposium on Theory of computing*, pages 169–178.
- Gilad-Bachrach, R., Dowlin, N., Laine, K., Lauter, K., Naehrig, M., and Wernsing, J. (2016). Cryptonets: Applying neural networks to encrypted data with high throughput and accuracy. In *International conference on machine learning*, pages 201–210. PMLR.
- Khan, T., Bakas, A., and Michalas, A. (2021). Blind faith: Privacy-preserving machine learning using function approximation. In *2021 IEEE Symposium on Computers and Communications (ISCC)*, pages 1–7.
- Khan, T., Nguyen, K., Michalas, A., and Bakas, A. (2023). Love or hate? share or split? privacy-preserving training using split learning and homomorphic encryption. In *2023 20th Annual International Conference on Privacy, Security and Trust (PST)*, pages 1–7. IEEE.
- Naehrig, M., Lauter, K., and Vaikuntanathan, V. (2011). Can homomorphic encryption be practical? In *Proceedings of the 3rd ACM workshop on Cloud computing security workshop*, pages 113–124.
- Nuuskala, C., Abdinasibfar, H., and Michalas, A. (2026). Spade: Digging into selective and partial decryption using functional encryption. In Alrabaee, S., Choo, K.-K. R., Damiani, E., and Deng, R. H., editors, *Security and Privacy in Communication Networks*, pages 34–58, Cham. Springer Nature Switzerland.
- Orekondy, T., Fritz, M., and Schiele, B. (2018). Connecting pixels to privacy and utility: Automatic redaction of private information in images. In *Proceedings of the IEEE Conference on Computer Vision and Pattern Recognition*, pages 8466–8475.
- Paillier, P. (1999). Public-key cryptosystems based on composite degree residuosity classes. In *International conference on the theory and applications of cryptographic techniques*, pages 223–238. Springer.
- Rivest, R. L., Adleman, L., Dertouzos, M. L., et al. (1978a). On data banks and privacy homomorphisms. *Foundations of secure computation*, 4(11):169–180.
- Rivest, R. L., Shamir, A., and Adleman, L. (1978b). A method for obtaining digital signatures and public-key cryptosystems. *Communications of the ACM*, 21(2):120–126.